

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №5
“Разработка простого бота для Telegram с использованием языка Python”

Выполнил:
Студент группы ИУ5-33Б
Якубова Д. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение разработки ботов в Telegram.

Задание лабораторной работы

Разработайте простого бота для Telegram. Бот должен использовать функциональность создания кнопок.

Листинг кода (доп задание):

```
bot.py
import datetime
import logging
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, MessageHandler,
CallbackQueryHandler, filters, ContextTypes
import requests
from config import TELEGRAM_TOKEN, OPENWEATHER_TOKEN

logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
logger = logging.getLogger(__name__)

code_to_smile = {
    "Clear": "Ясно ",
    "Clouds": "Облачно ",
    "Rain": "Дождь ",
    "Drizzle": "Морось ",
    "Thunderstorm": "Гроза ",
    "Snow": "Снег ",
    "Mist": "Туман ",
    "Smoke": "Дым ",
    "Haze": "Мгла ",
    "Dust": "Пыль ",
    "Fog": "Туман ",
    "Sand": "Песчаная буря ",
    "Ash": "Пепел ",
    "Squall": "Шквал ",
    "Tornado": "Торнадо "
}

# Столицы для быстрого выбора
CAPITALS = {
    "Москва": {"name": "Москва", "country": "RU"},
    "Лондон": {"name": "London", "country": "GB"},
    "Париж": {"name": "Paris", "country": "FR"}
}

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    welcome_text = (
        f"Привет, {user.first_name}! 🤖\n\n"
        "Я бот погоды \n\n"
        "Вы можете:\n"
        "1. Отправить название любого города\n"
        "2. Использовать команду /weather [город]\n"
        "3. Выбрать одну из популярных столиц ниже 📌\n\n"
```

```

        "Доступные команды:\n"
        "/start - начать диалог\n"
        "/help - помощь\n"
        "/weather [город] - узнать погоду\n"
        "/capitals - быстрый выбор столиц\n"
        "/allcapitals - погода во всех столицах\n"
        "/about - о боте"
    )

    keyboard = [
        [InlineKeyboardButton("RU Москва", callback_data="capital_Москва")],
        [InlineKeyboardButton("GB Лондон", callback_data="capital_Лондон")],
        [InlineKeyboardButton("FR Париж", callback_data="capital_Париж")],
        [InlineKeyboardButton("🌐 Все столицы", callback_data="all_capitals")]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)

    await update.message.reply_text(welcome_text, reply_markup=reply_markup)

    async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
        help_text = (
            " Как пользоваться ботом:\n\n"
            "1. Просто отправьте название города текстом:\n"
            "   Например: Москва\n\n"
            "2. Или используйте команду:\n"
            "   /weather Москва\n\n"
            "3. Выберите столицу из списка:\n"
            "   /capitals - меню выбора столиц\n"
            "   /allcapitals - погода сразу во всех столицах\n\n"
            "4. Поддерживаются города на русском и английском:\n"
            "   Санкт-Петербург, New York, Berlin\n\n"
            "Доступные команды:\n"
            "/start - начать\n"
            "/help - эта справка\n"
            "/weather [город] - узнать погоду\n"
            "/capitals - выбор столиц\n"
            "/allcapitals - погода во всех столицах\n"
            "/about - информация о боте"
        )
        await update.message.reply_text(help_text)

    async def about_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
        about_text = (
            " Weather Bot v2.0\n\n"
            "Бот показывает текущую погоду в любом городе мира.\n"
            "Имеет быстрый доступ к популярным столицам.\n\n"
            "Доступные столицы:\n"
            "• RU Москва (Россия)\n"
            "• GB Лондон (Великобритания)\n"
            "• FR Париж (Франция)\n\n"
            "Использует данные OpenWeatherMap API.\n"

```

```

        "Разработчик: Ваше имя\n"
        "Источник данных: openweathermap.org"
    )
    await update.message.reply_text(about_text)

async def capitals_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Показать меню выбора столиц"""
    keyboard = [
        [InlineKeyboardButton("RU Москва", callback_data="capital_Москва")],
        [InlineKeyboardButton("GB Лондон", callback_data="capital_Лондон")],
        [InlineKeyboardButton("FR Париж", callback_data="capital_Париж")],
        [InlineKeyboardButton("👉 Все столицы", callback_data="all_capitals")]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)

    text = (
        " Выберите столицу:\n\n"
        "• RU Москва - Россия\n"
        "• GB Лондон - Великобритания\n"
        "• FR Париж - Франция\n\n"
        "Или нажмите 'Все столицы' для одновременного показа погоды."
    )

    await update.message.reply_text(text, reply_markup=reply_markup)

async def all_capitals_command(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    """Показать погоду сразу во всех столицах"""
    await update.message.chat.send_action(action="typing")

    text = " *Погода в столицах*\n—————\n"

    for capital_name, capital_info in CAPITALS.items():
        try:
            weather_data = await fetch_weather_data(capital_info["name"])
            if weather_data:
                city_text = format_weather_data(weather_data, capital_name)
                text += f"\n{city_text}\n—————"
        except Exception as e:
            logger.error(f"Ошибка получения погоды для {capital_name}: {e}")
            text += f"\n❌ Ошибка получения данных для
{capital_name}\n—————"

    await update.message.reply_text(text, parse_mode='Markdown')

async def button_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Обработка нажатий кнопок"""
    query = update.callback_query
    await query.answer()

    if query.data.startswith("capital_"):

```

```

        capital_name = query.data.replace("capital_", "")
        if capital_name in CAPITALS:
            await get_weather(query, CAPITALS[capital_name]["name"])

    elif query.data == "all_capitals":
        await show_all_capitals(query)

async def show_all_capitals(query):
    """Показать погоду во всех столицах"""
    await query.message.chat.send_action(action="typing")

    text = " *Погода в столицах*\n—————\n"

    for capital_name, capital_info in CAPITALS.items():
        try:
            weather_data = await fetch_weather_data(capital_info["name"])
            if weather_data:
                city_text = format_weather_data(weather_data, capital_name)
                text += f"\n{city_text}\n—————"
        except Exception as e:
            logger.error(f"Ошибка получения погоды для {capital_name}: {e}")
            text += f"\n❌ Ошибка получения данных для {capital_name}\n—————"

    await query.edit_message_text(text, parse_mode='Markdown')

async def weather_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not context.args:
        keyboard = [
            [InlineKeyboardButton("RU Москва", callback_data="capital_Москва")],
            [InlineKeyboardButton("GB Лондон", callback_data="capital_Лондон")],
            [InlineKeyboardButton("FR Париж", callback_data="capital_Париж")]
        ]
        reply_markup = InlineKeyboardMarkup(keyboard)

        await update.message.reply_text(
            "Пожалуйста, укажите город.\n"
            "Пример: /weather Москва\n"
            "Или просто отправьте название города текстом.\n\n"
            "Или выберите одну из столиц:",
            reply_markup=reply_markup
        )
        return

    city = ' '.join(context.args)
    await get_weather(update, city)

async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    city = update.message.text.strip()
    await get_weather(update, city)

```

```

async def fetch_weather_data(city: str):
    """Получить данные о погоде для города"""
    url = f"http://api.openweathermap.org/data/2.5/weather"
    params = {
        'q': city,
        'appid': OPENWEATHER_TOKEN,
        'units': 'metric',
        'lang': 'ru'
    }

    response = requests.get(url, params=params, timeout=10)
    data = response.json()

    if data.get('cod') != 200:
        return None

    return data

def format_weather_data(data, custom_name=None):
    """Отформатировать данные о погоде в читаемый текст"""
    city_name = custom_name or data['name']
    country = data['sys']['country']
    temp = data['main']['temp']
    feels_like = data['main']['feels_like']
    humidity = data['main']['humidity']
    pressure = data['main']['pressure']
    wind_speed = data['wind']['speed']
    weather_desc = data['weather'][0]['main']
    description = data['weather'][0]['description']

    weather_emoji = code_to_smile.get(weather_desc, "☁️")

    sunrise =
datetime.datetime.fromtimestamp(data['sys']['sunrise']).strftime('%H:%M')
    sunset =
datetime.datetime.fromtimestamp(data['sys']['sunset']).strftime('%H:%M')

    # Флаги для стран
    country_flags = {
        'RU': 'RU',
        'GB': 'GB',
        'FR': 'FR'
    }

    flag = country_flags.get(country, '🇺🇸')

    weather_info = (
        f"{flag} {city_name}\n"
        f" Температура: {temp:.1f}°C\n"
        f" Ощущается как: {feels_like:.1f}°C\n"
        f" Ветер: {wind_speed} м/с\n"

```

```

        f" Влажность: *{humidity}%*\n"
        f" Погода: {weather_emoji} {description.capitalize()}"
    )

    return weather_info

async def get_weather(update, city: str):
    try:
        if hasattr(update, 'message'):
            await update.message.chat.send_action(action="typing")
            message_func = update.message.reply_text
        else:
            await update.message.chat.send_action(action="typing")
            message_func = update.edit_message_text

        weather_data = await fetch_weather_data(city)

        if weather_data is None:
            await message_func(f"❌ Город '{city}' не найден. Проверьте написание.")
            return

        # Получаем русское название для столиц, если нужно
        display_name = city
        for cap_name, cap_info in CAPITALS.items():
            if cap_info["name"].lower() == city.lower():
                display_name = cap_name
                break

        weather_info = format_weather_data(weather_data, display_name)

        full_info = f"{weather_info}\n—————\nХорошего дня! 🌟"

        await message_func(full_info, parse_mode='Markdown')

    except requests.exceptions.Timeout:
        await message_func("🕒 Превышено время ожидания ответа от сервера погоды.")
    except requests.exceptions.ConnectionError:
        await message_func("🔌 Ошибка подключения к интернету.")
    except Exception as e:
        logger.error(f"Ошибка: {e}")
        await message_func("❌ Произошла ошибка. Попробуйте позже.")

async def error_handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
    logger.error(f"Ошибка: {context.error}")
    try:
        if update and update.message:
            await update.message.reply_text("⚠️ Произошла ошибка. Попробуйте снова.")
    except:

```



```

pass

def main():
    if TELEGRAM_TOKEN == "ВАШ_TELEGRAM_TOKEN_ЗДЕСЬ":
        print("❌ ОШИБКА: Укажите Telegram токен!")
        print("1. Создайте файл config.py с содержанием:")
        print("    TELEGRAM_TOKEN = 'ваш_токен'")
        print("    OPENWEATHER_TOKEN = 'ваш_токен'")
        print("\n2. Или замените токены прямо в этом файле")
        print("\n3. Получите токены:")
        print("    - Telegram: @BotFather")
        print("    - OpenWeather: openweathermap.org")
        return

    if OPENWEATHER_TOKEN == "ВАШ_OPENWEATHER_TOKEN_ЗДЕСЬ":
        print("❌ ОШИБКА: Укажите OpenWeather токен!")
        return

    application = Application.builder().token(TELEGRAM_TOKEN).build()

    # Регистрация обработчиков команд
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("help", help_command))
    application.add_handler(CommandHandler("weather", weather_command))
    application.add_handler(CommandHandler("about", about_command))
    application.add_handler(CommandHandler("capitals", capitals_command))
    application.add_handler(CommandHandler("allcapitals", all_capitals_command))

    # Обработчик кнопок
    application.add_handler(CallbackQueryHandler(button_callback))

    # Обработчик текстовых сообщений
    application.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,
    handle_text))

    # Обработчик ошибок
    application.add_error_handler(error_handler)

    print("=" * 50)
    print("✅ Бот запускается...")
    print(" Проверьте бота в Telegram")
    print(" Доступны быстрые столицы: Москва, Лондон, Париж")
    print(" Для остановки нажмите Ctrl+C")
    print("=" * 50)

    application.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == '__main__':
    main()

```

Скриншот работы программы

